

Plano — Alarmes que refletem o estado real + Visão geral de KPIs que se atualiza sozinha

Data: 2026-09-04 **Origem:** diagnóstico de dois problemas relatados na operação — (a) o painel “Ver KPIs” (9 gráficos) só atualiza quando o usuário alterna 4G/5G ou fecha e reabre; (b) alarmes já limpos no iManager continuam aparecendo no drawer de alarmes. **Decisão de produto (chefe, 2026-09-04):** a tabela de alarmes deve ser **zerada ao fechar o aplicativo**, e o painel deve mostrar **apenas os alarmes realmente ativos**.

Por que três fases

As duas correções são independentes entre si (uma é backend de alarmes, a outra é frontend de gráficos) e a de alarmes tem duas metades com riscos e superfícies de teste bem diferentes:

Fase	Entrega	Arquivos tocados	Risco
1	O alarme limpo desaparece do painel durante o evento	<code>database.py</code> , <code>collector.py</code> , <code>scheduler.py</code> , <code>api.py</code> , <code>event_export.py</code>	Médio — mexe em schema e no ciclo de coleta
2	A tabela de alarmes é zerada ao fechar o app	<code>main.py</code> , <code>database.py</code> , <code>api.py</code>	Baixo — isolado no encerramento
3	O painel de 9 gráficos se atualiza a cada ciclo	<code>kpi_overview.js</code> , <code>app.js</code> , <code>index.html</code>	Baixo — só frontend

A Fase 2 sozinha não resolve o problema relatado. Zerar ao fechar só garante que a *próxima* sessão comece limpa; durante um evento de várias horas com o app aberto, um alarme limpo no iManager continuaria na tela até o fim. É a Fase 1 que entrega “só os realmente ativos”. As duas juntas cobrem o pedido completo, mas separá-las mantém cada commit pequeno, revisável e reversível sozinho.

A Fase 3 não depende de nenhuma das outras e pode ser feita em paralelo ou primeiro, se preferir entregar antes o alívio mais visível para quem opera.

FASE 1 — O alarme limpo sai do painel

O que será desenvolvido (explicação simples)

Hoje o app guarda todo alarme que já viu e nunca tira nenhum. A resposta do iManager traz o campo `cleared` (0 ou 1) junto de `clearUtc` , mas o app **descarta esses campos na ingestão** e a tabela nem tem coluna para eles. Além disso a gravação é `INSERT OR IGNORE` por `csn` : se o mesmo alarme voltar depois já limpo, a atualização é ignorada e a linha fica congelada como estava. O resultado é uma lista que só cresce.

Esta fase ensina o app três coisas: **guardar** o estado de limpeza que já vem na resposta, **atualizar** um alarme que ele já conhece, e **concluir** que um alarme sumido da coleta foi limpo. A leitura passa a devolver só os ativos.

Escopo detalhado

No código

core/database.py

1. Schema da tabela `alarms` (`CREATE TABLE` , hoje em `database.py:146`): três colunas novas.

```
cleared    INTEGER NOT NULL DEFAULT 0,
clear_time TEXT,
acked      INTEGER NOT NULL DEFAULT 0
```

2. Migração aditiva para bancos de eventos já existentes, no mesmo bloco onde `kpi_measurements` já é migrada (`database.py:167-171`), usando o mesmo padrão `PRAGMA table_info` + `ALTER TABLE` . É idempotente e não exige bump de `_EVENT_DB_SCHEMA_VERSION` .
3. Índice de leitura: `CREATE INDEX IF NOT EXISTS idx_alarms_cleared ON alarms(event_id, cleared)` .
4. `insert_alarms_batch` — trocar `INSERT OR IGNORE` por upsert, para que a coleta do mesmo `csn` atualize o estado:

```
INSERT INTO alarms (...) VALUES (...)
ON CONFLICT(csn) DO UPDATE SET
    severity = excluded.severity,
    cleared = excluded.cleared,
    clear_time = excluded.clear_time,
    acked = excluded.acked,
    additional_info = excluded.additional_info,
    collected_at = excluded.collected_at
```

O retorno da função continua `None` — `_insert_count` (`scheduler.py:242`) já trata isso e a semântica de `result.inserted` não muda.

5. `reconcile_alarms(event_id, active_csns, cleared_at) -> int` — nova função. Marca `cleared = 1`, `clear_time = :cleared_at` em toda linha do evento que hoje está `cleared = 0` e não está no conjunto de `csn` do ciclo.
⚠ Não usar `NOT IN (?, ?, ...)` com a lista inline. O filtro real devolve milhares de linhas (6.095 nos HAR de referência) e isso esbarra no limite de variáveis do SQLite. Usar uma tabela temporária: `CREATE TEMP TABLE IF NOT EXISTS active_csn(csn INTEGER PRIMARY KEY)` , `DELETE FROM active_csn` , executemany do lote, e então `UPDATE alarms SET ... WHERE event_id = ? AND cleared = 0 AND csn NOT IN (SELECT csn FROM active_csn)` .
6. `get_alarms(event_id, timestamp=None, limit=500)` — passa a filtrar por estado:
 - o **Modo ativo**: `WHERE event_id = ? AND cleared = 0`
 - o **Modo histórico (timeline)**: `WHERE event_id = ? AND collected_at <= ? AND (clear_time IS NULL OR clear_time > ?)` — ou seja, “o que estava ativo naquele instante”. Isso corrige de quebra a timeline, que hoje mostra tudo que já foi coletado até o ponto, alarmes já limpos incluídos.
7. `delete_alarms_by_names(event_id, names)` — nova função pequena, usada pelo filtro (item 16).

core/collector.py

8. `_flatten_alarm` (`collector.py:215`) — mapear os campos que já chegam na resposta:

```
"cleared":    int(bool(a.get("cleared"))),
"clear_time": a.get("clearUtc"),
"acked":      int(bool(a.get("acked"))),
```

9. `_flatten_alarms` (`collector.py:2891`) — converter `clear_time` com `_parse_trace_timestamp`, igual já é feito com `arrive_time` e `occur_time` (os campos "Utc" chegam no fuso do cliente).
10. `_collect_alarm_pages` (`collector.py:2873`) — passar a devolver (rows, complete), com `complete = len(rows) >= total`. O `break` do laço quando uma página volta vazia hoje produz um lote **parcial** indistinguível de um completo.
11. `collect_alarms` (`collector.py:2753`) — propagar a integridade em `result.coverage["complete"] = complete`, tanto no caminho `CollectionResult.data` quanto no `CollectionResult.empty`.
12. `_ALARM_SEVERITY` (`collector.py:104-105`) — corrigir 5 e 6. As constantes reais do FM (extraídas do `app_main.js` capturado nos HAR) são {1:CRITICAL, 2:MAJOR, 3:MINOR, 4:WARNING, 5:EVENT, 6:ALARM_NAME}. O mapa atual diz 5:"Indeterminate", 6:"Cleared" — a string "Indeterminate" não existe nesse sistema e **não existe severidade "Cleared"** (limpeza é o campo `cleared`, não a severidade). Impacto prático é nulo hoje porque `alarmLevel` só pede CRITICAL/MAJOR/MINOR/WARNING, mas o 6: "Cleared" é exatamente o que dá a impressão falsa de que o clear já estava tratado.

core/scheduler.py

13. `_apply_result` (`scheduler.py:250`) — novo parâmetro opcional `finalize: Optional[Callable] = None`, invocado **dentro do** `_state_lock`, depois do `persist`, e **somente** quando `result.state in {"data", "empty"}`.
⚠ Tem que rodar também com `measurements` vazio. O bloco atual só chama `persist` sob `if measurements:`, e "nenhum alarme ativo" é justamente o caso em que a reconciliação mais importa — sem isso, limpar o último alarme deixaria a lista antiga na tela para sempre.
14. `_collect_alarms` (`scheduler.py:395`) — montar `active = {m["csn"] for m in result.measurements}` e passar `finalize` chamando `db.reconcile_alarms(...)`, **apenas se** `result.coverage.get("complete")`. Ciclo parcial nunca reconcilia (senão uma paginação interrompida limparia o painel inteiro).

api/api.py

15. `get_alarms` (`api.py:2398`) — sem mudança de assinatura; a filtragem desce para o `db`. Só conferir que as chaves novas no `dict` da linha não atrapalham `_resolve_site_for_source`.
16. `set_alarm_filter` (`api.py:2550`) — antes de `collect_alarms_now()`, apagar do banco os alarmes cujos tipos saíram da seleção, via `db.delete_alarms_by_names`.
Sem isso, a reconciliação marcaria os tipos desmarcados como "limpos", o que é semanticamente errado (eles não foram limpos, foram desselecionados) e sujaria o `clear_time`. É um órfão criado por esta mudança, então é escopo dela.

core/event_export.py

17. `_export_alarms` (`event_export.py:929`) — acrescentar `cleared`, `clear_time` e `acked` a `raw_headers`. O `sinks.write` itera sobre `headers` com `row.get(...)`, então colunas novas na tabela não quebram nada — mas sem entrar no header elas não saem no CSV. **Decisão:** exportar **todas** as linhas (ativas e limpas). O CSV é registro forense do evento; a coluna `cleared` diz o estado. Só o painel filtra.

frontend/js/alarms.js

18. Corrigir o comentário de `alarms.js:274` — “*não acumula: é snapshot corrente*”. Descreve a intenção, mas a premissa era falsa. Depois desta fase passa a ser verdadeira; vale reescrever dizendo **por que** (o backend devolve só ativos).

Na interface

Sem elemento novo. As mudanças são de conteúdo:

- O drawer passa a listar só alarmes ativos.
- O resumo “X no evento · Y na rede” passa a contar só ativos.
- O badge de críticos no header cai sozinho quando o alarme é limpo no iManager.
- Desmarcar um tipo no filtro remove os alarmes daquele tipo da lista (hoje eles ficam para sempre).

Nos testes

Em `tests/test_alarms.py`:

Teste	O que prova
<code>test_flatten_captura_cleared_e_clear_time</code>	Os campos param de ser descartados na ingestão
<code>test_upsert_atualiza_alarme_ja_conhecido</code>	Mesmo <code>csn</code> com <code>cleared</code> 0→1 atualiza a linha
<code>test_reconcile_marca_ausentes_como_limpos</code>	Sumir da coleta ⇒ <code>cleared=1</code> + <code>clear_time</code>
<code>test_reconcile_preserva_os_presentes</code>	Quem continua no ciclo não é marcado
<code>test_reconcile_com_lote_grande</code>	5.000 <code>csn</code> ativos não estouram o limite de variáveis
<code>test_get_alarms_devolve_so_ativos</code>	A leitura filtra <code>cleared = 0</code>
<code>test_get_alarms_historico_mostra_o_que_estava_ativo</code>	Timeline usa <code>clear_time > timestamp</code>
<code>test_migracao_aditiva_em_banco_legacy</code>	Banco sem as colunas abre e ganha as três
<code>test_severidade_5_e_6_seguem_o_catalogo_do_fm</code>	Corrige <code>Indeterminate</code> / <code>Cleared</code>
<code>test_set_alarm_filter_remove_tipos_desmarcados</code>	Filtro apaga em vez de marcar como limpo

Em `tests/test_scheduler.py`:

Teste	O que prova
<code>test_ciclo_completo_reconcilia</code>	<code>finalize</code> roda com <code>state="data"</code> e <code>complete=True</code>
<code>test_ciclo_vazio_reconcilia</code>	<code>state="empty"</code> também reconcilia (todos limpos)
<code>test_ciclo_incompleto_nao_reconcilia</code>	Paginação interrompida não limpa a tela
<code>test_ciclo_com_erro_nao_reconcilia</code>	<code>error</code> / <code>auth_required</code> não tocam no banco

Em `tests/test_event_export.py`: `test_export_alarmes_inclui_colunas_de_clear`.

Como validar e testar

Testes automatizados

```
python -m pytest tests/test_alarms.py tests/test_scheduler.py tests/test_event_export.py -v
```

Depois a suíte inteira, para garantir que a mudança de schema não quebrou nada:

```
python -m pytest -q
```

Validação visual (com VPN do cliente)

1. Iniciar um evento e abrir o drawer de alarmes. Anotar um alarme específico (`source` + nome) e o número no badge de críticos.
2. Limpar esse alarme no iManager.
3. Esperar o próximo ciclo de FM (≤ 3 min, `INTERVAL_ALARMS_SECONDS = 180`) ou clicar no botão de atualizar do drawer para forçar.
4. **Esperado:** o alarme some da lista, o resumo "X no evento" diminui e, se ele era crítico, o badge do header cai — sem fechar nem reabrir nada.
5. Conferir no banco que ele foi *marcado*, não apagado:

```
sqlite3 data/events/<evento>.db "SELECT csn, alarm_name, cleared, clear_time FROM alarms WHERE cleared = 1 LIMIT 5;"
```

6. **Timeline:** entrar no modo histórico e arrastar o slider para um instante **anterior** à limpeza. O alarme deve reaparecer ali (estava ativo naquele momento) e sumir de novo nos pontos posteriores.
7. **Filtro:** desmarcar um tipo de alarme e confirmar que os daquele tipo somem da lista.

Validação em modo mock

```
python main.py --mock
```

Confirma que a migração roda e que a tela não quebra sem OSS.

Sugestão de commit

```
feat: marcar alarme limpo e remover do painel
```

FASE 2 — Zerar os alarmes ao fechar o aplicativo

O que será desenvolvido (explicação simples)

Decisão do chefe: ao fechar o app, a tabela de alarmes é esvaziada, para que a próxima sessão comece do zero e colete os mais recentes. Isso elimina qualquer chance de arrastar alarme velho entre sessões, e mantém o banco do evento enxuto.

Vale junto com a Fase 1, não no lugar dela: a Fase 1 cuida do evento **em andamento**, esta cuida da **fronteira entre sessões**.

Escopo detalhado

No código

core/database.py

1. `clear_alarms(event_id) -> int` — `DELETE FROM alarms WHERE event_id = ?`, com commit. Devolve quantas linhas saíram (útil para o log).
2. `clear_all_alarms() -> int` — percorre os eventos locais (`get_events()`) e chama a função acima para cada um, somando. Alarme de um evento antigo que ficou para trás também sai.

main.py

3. `_ShutdownCoordinator` (`main.py:84`) — receber no construtor um callable de limpeza (ex.: `purge_alarms=db.clear_all_alarms`), do mesmo jeito que já recebe `scheduler_obj`, `chrome_controller` e `timer_factory`.

Injetar em vez de importar direto mantém o padrão dos testes existentes (`_FakeScheduler`, `_FakeChrome` em `tests/test_main_shutdown.py`) e evita I/O de banco em teste unitário. O `main.py` já importa `core.database as db` na linha 45.

4. `shutdown()` (`main.py:106`) — chamar a limpeza **depois** de `self._scheduler.stop()`, nunca antes.

⚠ A ordem não é detalhe: um ciclo de coleta em voo regravaria a tabela logo após o `DELETE`. `stop()` sinaliza o `stop_event` dos workers antes de retornar.

Envolver em `try/except` com `logger.exception`, no mesmo estilo dos dois blocos que já existem ali. **Falha na limpeza não pode impedir o fechamento da janela**. A idempotência já vem de graça pelo flag `_complete`, que cobre as duas chamadas (`_on_closing` em `main.py:638` e o `finally` em `main.py:663`).

api/api.py

5. `activate_event` (`api.py:139`) — chamar `db.clear_alarms(event_id)` antes de `scheduler.start(config, mock=mock)`.

Rede de segurança para o encerramento anormal: se o app for morto ou cair, o `shutdown()` nunca roda e a tabela sobrevive. Como o objetivo declarado é “obter os mais recentes”, limpar também na ativação é o que torna a garantia real em vez de otimista. São 2 linhas. Se preferir ficar estritamente no que foi pedido, este item pode ser cortado sem afetar os demais.

Na interface

Nada muda durante o uso. A única diferença perceptível: ao reabrir o app, o drawer começa **vazio** e se preenche no primeiro ciclo de alarmes (`ALARMS_INITIAL_DELAY_SECONDS = 10` s após o start, depois a cada 3 min). Vazio por alguns segundos é o comportamento correto — melhor que mostrar dado velho.

Nos testes

Em `tests/test_main_shutdown.py` (estende os fakes já existentes com um `_FakePurge` que conta chamadas e registra a ordem):

Teste	O que prova
<code>test_shutdown_limpa_alarmes_depois_de_parar_o_scheduler</code>	Ordem correta: <code>stop()</code> antes do <code>DELETE</code>
<code>test_shutdown_nao_limpa_duas_vezes</code>	Idempotência via <code>_complete</code>
<code>test_shutdown_conclui_mesmo_se_a_limpeza_falhar</code>	Exceção na limpeza não trava o fechamento

Em `tests/test_alarms.py`:

Teste	O que prova
<code>test_clear_alarms_esvazia_so_o_evento_pedido</code>	Escopo por <code>event_id</code>
<code>test_clear_all_alarms_percorre_os_eventos_locais</code>	Varredura completa
<code>test_clear_alarms_preserva_kpis_e_alertas</code>	Não é um <code>clear_event_history</code> disfarçado

Em `tests/test_api.py`: `test_activate_event_zera_alarmes_residuais`.

Como validar e testar

Testes automatizados

```
python -m pytest tests/test_main_shutdown.py tests/test_alarms.py tests/test_api.py -v
```

Validação visual

1. Com o app aberto e alarmes visíveis no drawer, conferir a contagem:

```
sqlite3 data/events/<evento>.db "SELECT COUNT(*) FROM alarms;"
```

2. Fechar o app pelo X da janela.
3. Rodar a mesma consulta: deve devolver 0.
4. Reabrir o app e o evento. O drawer começa vazio e volta a se preencher em até ~3 min.
5. **Encerramento anormal** (valida o item 5): reabrir, deixar coletar alarmes, matar o processo pelo Gerenciador de Tarefas, conferir que a tabela **não** está vazia, reabrir o app e ativar o evento — a tabela deve zerar na ativação.
6. Confirmar que os KPIs **não** foram apagados junto:

```
sqlite3 data/events/<evento>.db "SELECT COUNT(*) FROM kpi_measurements;"
```

Sugestão de commit

```
feat: zerar alarmes coletados ao fechar o aplicativo
```

FASE 3 — A visão geral de KPIs se atualiza sozinha

O que será desenvolvido (explicação simples)

O painel de 9 gráficos não tem nenhum refresh periódico. O `_load()` só dispara em quatro ações do usuário: abrir o painel, trocar 4G/5G, trocar a janela de tempo e mudar a seleção. É exatamente por isso que alternar 4G↔5G ou fechar e reabrir “resolve” — são dois desses quatro gatilhos.

O dashboard já tem um poll de 120 s que, no fim, chama `refreshChart()` para o gráfico único. Esta fase pendura o painel de 9 gráficos nesse mesmo poll — sem criar timer novo, mantendo as duas telas em fase e sem carga extra fora de ciclo. O refresh precisa ser **silencioso**: recarregar do jeito atual piscaria um overlay opaco sobre a grade inteira a cada 2 minutos.

Escopo detalhado

No código

`frontend/js/kpi_overview.js`

1. `_load(options = {})` com `{ silent = false }` (hoje em `kpi_overview.js:1019`):
 - quando `silent`, não chamar `_showState("loading")`. O `.kpi-overview-state` é um overlay opaco (`rgba(13,17,23,0.88)`, `main.css:1526`) cobrindo os nove cards.
 - quando `silent` e a requisição falhar: manter os gráficos que estão na tela e apenas `console.error`. Trocar o painel inteiro por tela de erro num refresh de fundo é pior que o dado ficar 2 minutos velho.
 - o guarda de corrida por `_requestId` continua igual.
2. `_renderCharts(response, { reuse = false })` (`kpi_overview.js:1048`): quando a família e o conjunto de escopos não mudaram, atualizar `chart.data.labels` / `chart.data.datasets` e chamar `chart.update("none")`, em vez de `_destroyCharts()` + recriar as nove instâncias.

`_destroyCharts` (`kpi_overview.js:979`) zera `_activeIndex` e `_hoveredPanelId` — num refresh automático isso mataria o crosshair e o tooltip debaixo do mouse do usuário.

3. `export function refreshKpiOverview()` — ponto de entrada do poll. Retorna cedo (sem fazer nada) se qualquer destas for verdadeira:
 - `!_isOpen()` — painel fechado;
 - `State.mode !== "active"` — modo histórico é congelado por definição;
 - existe `.scope-picker.open` — o usuário está montando a comparação;
 - `_hoveredPanelId !== null` — o mouse está lendo um card.

Caso contrário: `void _load({ silent: true })`.

4. **Carimbo de atualização** — guardar o horário de cada carga bem-sucedida e escrever em `#kpi-overview-updated`.

`frontend/js/app.js`

5. Importar `refreshKpiOverview` (junto de `initKpiOverview` e `resizeKpiOverviewCharts`, `app.js:12`) e chamá-la dentro de `_poll`, ao lado de `refreshChart()` (`app.js:326`).
- `_poll` já retorna cedo quando `State.mode !== "active"` (`app.js:293`), então o modo timeline continua congelado sem nenhum código a mais.

frontend/index.html

6. `<span id="kpi-overview-updated"></span>` dentro de `.kpi-overview-heading` (ao lado de `#kpi-overview-context`, `index.html:676`).
7. Bump dos `?v=`: no import de `kpi_overview.js` em `app.js:12` e no `<script>` de `app.js` em `index.html:725`.
Convenção do projeto para furar o cache do WebView2 — o `ERRORS.md` registra um caso de código correto em disco que não aparecia na tela por causa disso.

frontend/css/main.css

8. Estilo do `#kpi-overview-updated` seguindo `#kpi-overview-context` (secundário, discreto).

Na interface

- Os nove gráficos andam sozinhos a cada 120 s, **sem piscar** e sem overlay.
- O crosshair e o tooltip sob o mouse sobrevivem ao refresh.
- Um seletor aberto não é atropelado — a atualização espera o próximo ciclo.
- Carimbo "atualizado às HH:MM" no cabeçalho: hoje não há como saber se o que está na tela tem 1 ou 40 minutos.
- Em modo histórico, nada muda (continua congelado, como deve).

Nos testes

Em `tests/test_frontend_kpi_overview_ui.py` (mesmo padrão já usado: servidor estático + Playwright sobre os mocks de `bridge.js`; o refresh é disparado do teste via `await import('./js/kpi_overview.js?v=...')` seguido de `mod.refreshKpiOverview()` — o import dinâmico devolve a mesma instância do módulo já carregado):

Teste	O que prova
<code>test_refresh_silencioso_nao_mostra_overlay_de_carregamento</code>	<code>#kpi-overview-loading</code> fica <code>hidden</code> durante o refresh
<code>test_refresh_preserva_crosshair_e_selecao</code>	<code>_activeIndex</code> e os escopos sobrevivem
<code>test_refresh_nao_roda_com_seletor_aberto</code>	Nenhuma requisição com <code>.scope-picker.open</code>
<code>test_refresh_ignorado_com_painel_fechado</code>	Fechado ⇒ não busca nada
<code>test_carimbo_de_atualizacao_muda_apos_refresh</code>	O horário no cabeçalho avança
<code>test_poll_do_dashboard_chama_o_refresh_da_visao_geral</code>	Inspeção de fonte em <code>app.js</code> (padrão já usado em <code>test_paleta_de_series_nao_e_reordenada_sem_revalidar</code>)

Como validar e testar

Testes automatizados

```
python -m pytest tests/test_frontend_kpi_overview_ui.py -v
python -m pytest tests/test_frontend_kpi_chart_ui.py -v # garante que o gráfico único não regrediu
```

Validação visual (mock, rápido)

```
python main.py --mock --dev
```

1. Abrir “Ver KPIs”, escolher a janela de **15 min**.
2. Deixar aberto ~5 minutos, com o DevTools no console (não deve haver erro).
3. **Esperado**: o eixo X anda sozinho, sem piscar e sem a faixa “Carregando os nove painéis...”.
4. Passar o mouse sobre um card e **deixar parado** durante um ciclo: o tooltip e o crosshair não podem sumir.
5. Abrir o dropdown de Sites e deixar aberto durante um ciclo: a lista não pode ser atropelada.
6. Fechar o painel, esperar dois ciclos, reabrir: sem erro no console (nada rodando às cegas).

Validação visual (produção, com VPN)

7. Com o painel aberto, comparar o último ponto de um KPI com o mesmo KPI no gráfico do dashboard — depois de um ciclo, os dois têm que bater. Hoje o do painel fica para trás.
8. Entrar no modo histórico com o painel aberto e confirmar que ele **não** se move.
9. Conferir o carimbo “atualizado às HH:MM” avançando a cada ~2 min.

Sugestão de commit

```
feat: atualizar a visão geral de KPIs a cada ciclo de coleta
```

Ordem sugerida de execução

1. **Fase 1** — é o bug que dói na operação (alarme fantasma orienta decisão errada em campo).
2. **Fase 2** — pequena e fecha o pedido do chefe.
3. **Fase 3** — independente; pode ser antecipada se o incômodo do painel for mais urgente.

Evidências que sustentam o plano

- Campos `cleared`, `clearUtc`, `clearType`, `clearUser`, `acked`, `status` confirmados nas respostas do `cmd 1103` em `alarms/10.220.50.9.new.har` (2.664 linhas inspecionadas).
- Constantes do FM extraídas do `app_main.js` capturado no HAR: `alarmStatus: {ACKED_CLEARED:10, ACKED_UNCLEARED:11, UNACKED_CLEARED:12, UNACKED_UNCLEARED:13}` e `alarmSeverityInt: {CRITICAL:1, MAJOR:2, MINOR:3, WARNING:4, EVENT:5, ALARM_NAME:6}`.
- A `condition` enviada pelo coletor (`collector.py:110`) é `alarmStatus: [12, 10, 11, 13]` — as quatro combinações, **cleared inclusive**. A consulta já pede os alarmes limpos; o app é que não sabia o que fazer com eles.
- O standalone `imaster_alarms.py:352` já capturava `cleared`; o campo se perdeu na portagem para `core/collector.py`.
- Nenhum teste em `tests/test_alarms.py` menciona `clear/cleared` — confirma que nunca foi implementado, e que não há regressão escondida a temer.